FinHedge AI

User Manual & Deployment Guide

DA5402 - MLOps in Practice
Indian Institute of Technology Madras
April 2026

Author: Siddharth Chauhan
Roll No.: CH21B103
Course: DA5402 MLOps
Project: FinHedge AI - Stock Prediction & Hedging Platform

This manual covers: prerequisites, one-command deployment, using each application page, running the ML pipeline, monitoring, and common troubleshooting steps.

0 Contents

1 Quick-Start in 5 Minutes **2**

2 Application Pages Walkthrough **2**
- 2.1 Page 1 - Dashboard (Home) . . . 3
- 2.2 Page 2 - Predictions . . . 3
- 2.3 Page 3 - Hedging Strategy . . . 4
- 2.4 Page 4 - Pipeline Management . . . 5
- 2.5 Page 5 - Monitoring . . . 6

3 Running the ML Pipeline from CLI **7**
- 3.1 Using the REST API (curl) . . . 8
- 3.2 Using the FastAPI Swagger UI . . . 8

4 MLflow Experiment Tracking **9**

5 Grafana Monitoring Dashboards **10**

6 Apache Airflow - Scheduled Pipeline **11**

7 Stopping and Restarting the Application **12**

8 Troubleshooting **13**
- 8.1 Checking Service Health . . . 13

9 CI/CD with GitHub Actions **14**

10 Adding a New Ticker **14**

11 Environment Configuration **15**

Appendix - Keyboard Shortcuts **16**

1 Quick-Start in 5 Minutes

Step 1. Prerequisites - Ensure the following are installed:

- Docker Desktop $\geq$ 24.0 (Windows / macOS / Linux)
- Docker Compose v2 (`docker compose` command)
- Git $\geq$ 2.40
- 8 GB RAM and 10 GB free disk space recommended

Step 2. Clone the repository

```
git clone https://github.com/<your-username>/finhedge.git
cd finhedge
```

Step 3. Start all services

```
docker compose up -d
```

This pulls images and starts 8 containers. First run takes 3–5 minutes.

Step 4. Verify containers are healthy

```
docker compose ps
```

All services should show `Up` or `healthy`.

Step 5. Open the application - navigate to any of the URLs in Table 1.

Table 1: Service URLs after `docker compose up`

Service	URL	Username	Password
Streamlit App (main UI)	http://localhost:8501	-	-
FastAPI Docs	http://localhost:8000/docs	-	-
MLflow Tracking	http://localhost:5000	-	-
Prometheus	http://localhost:9090	-	-
Grafana Dashboards	http://localhost:3000	admin	finhedge123
Airflow Scheduler	http://localhost:8080	admin	admin

Tip

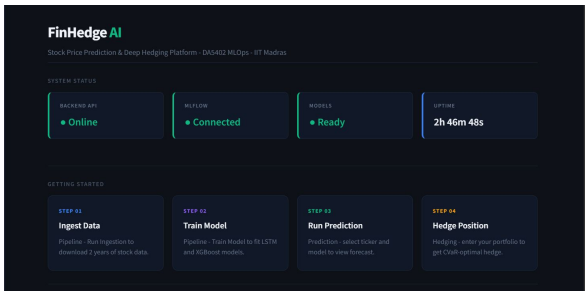
On Windows, use `http://127.0.0.1:PORT` if `localhost` does not resolve. Grafana may also be on port `3001` if 3000 is already in use - check `docker compose ps`.

2 Application Pages Walkthrough

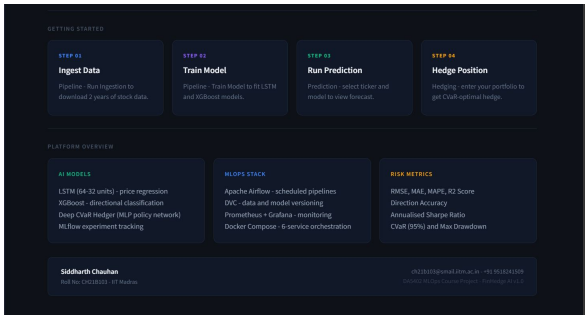
The Streamlit frontend has five pages accessible from the left sidebar.

2.1 Page 1 - Dashboard (Home)

The dashboard is the landing page showing the system health overview.



(a) Dashboard overview



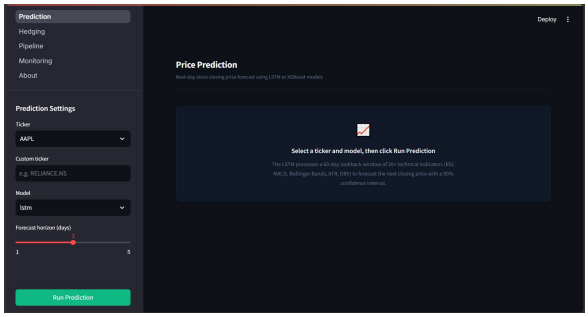
(b) Stock chart with predictions

Figure 1: Page 1 - Dashboard

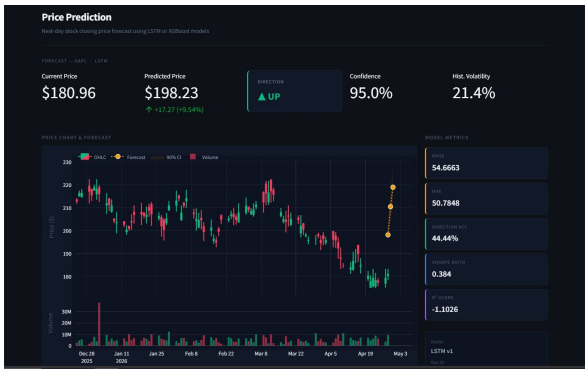
What you can do here:

- View OHLCV candlestick chart for any ticker
- See the latest model prediction vs. actual price
- Check API and pipeline health status
- Select a ticker from the sidebar and click **Load Data**

2.2 Page 2 - Predictions



(a) Prediction form



(b) Prediction result with confidence

Figure 2: Page 2 - Predictions

How to make a prediction:

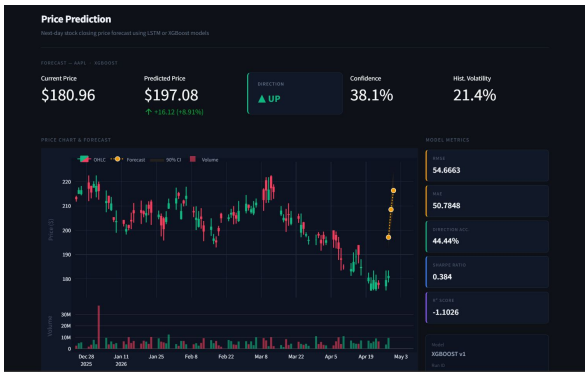
1. Enter a ticker symbol (e.g., `AAPL`, `MSFT`) in the sidebar
2. Select model type: **LSTM** or **GRU**
3. Click **Predict Next Close Price**
4. The predicted closing price and confidence interval appear on the right

5. Scroll down to view the historical prediction vs. actual chart

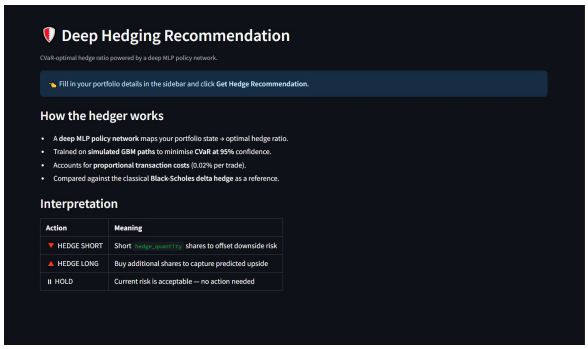
Note

A model must be trained for the ticker before predictions work. If you see "Model not found", go to **Page 3 - Pipeline** and run a training job first.

2.3 Page 3 - Hedging Strategy



(a) Hedging strategy recommendation



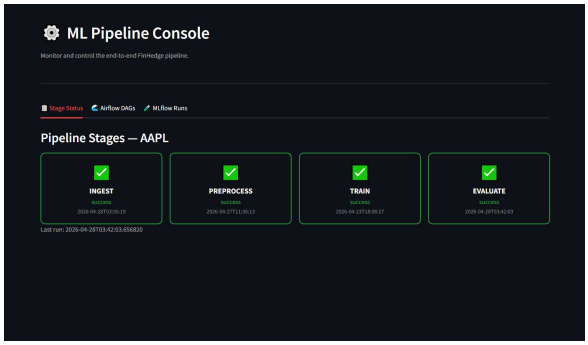
(b) P&L and risk metrics

Figure 3: Page 3 - Hedging Strategy

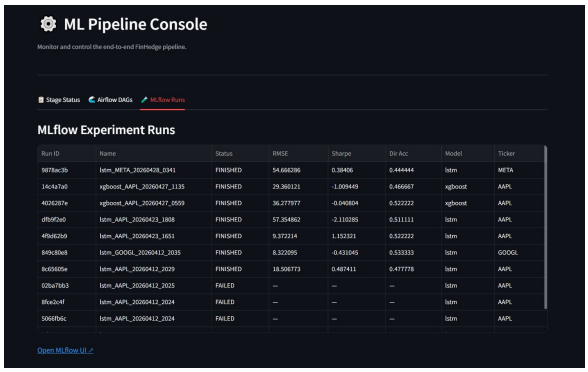
Using the hedging tool:

1. Enter ticker, portfolio value (USD), and position size
2. Select risk tolerance: **Low** / **Medium** / **High**
3. Click **Get Hedge Recommendation**
4. The system outputs:
   - Action: **HEDGE_LONG**, **HEDGE_SHORT**, or **HOLD**
   - Confidence score and recommendation rationale
   - P&L simulation chart over the next 30 days
   - Risk metrics: VaR, Max Drawdown, Sharpe Ratio

2.4 Page 4 - Pipeline Management

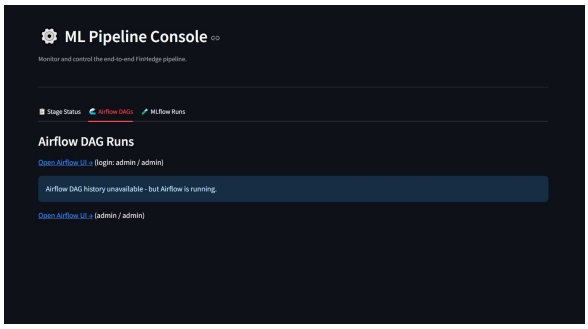


(a) Pipeline stage status

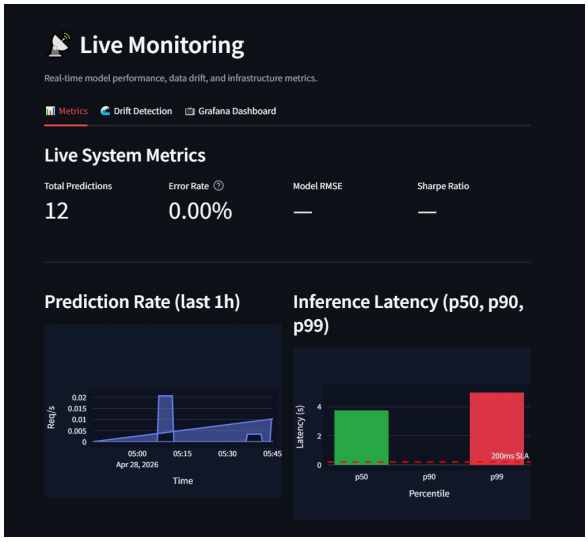


(b) Training configuration panel

Figure 4: Page 4 - Pipeline Management



(a) MLflow experiment runs list



(b) Training metrics after run

Figure 5: Page 4 - Pipeline and MLflow integration

Running the full pipeline:

1. Enter a ticker in the sidebar (default: `AAPL`)
2. Click **Run Full Training Pipeline** to execute all four stages sequentially:
   a. **Ingest** - downloads OHLCV data from Yahoo Finance
   b. **Preprocess** - builds features, fits scaler
   c. **Train** - trains LSTM/GRU, logs run to MLflow
   d. **Evaluate** - saves metrics to `metrics/eval_metrics.json`
3. Stage status indicators turn green as each stage completes
4. Scroll down to view the MLflow runs table with RMSE, Sharpe, and direction accuracy

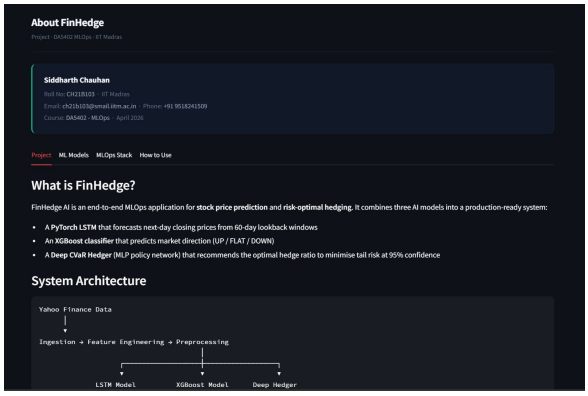
Running individual stages:

1. Use the **Trigger Stage** expander
2. Select the stage from the dropdown (Ingest / Preprocess / Train / Evaluate)
3. Click **Trigger** - the job runs asynchronously
4. Poll the job status with the displayed job ID

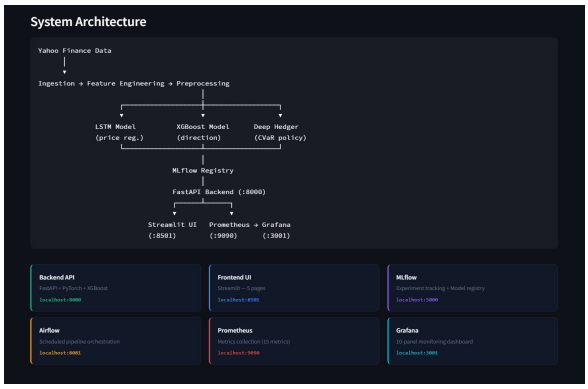
Warning

Training is CPU-intensive and may take 2–10 minutes depending on the number of epochs and your hardware. The UI will show a spinner while training is in progress.

2.5 Page 5 - Monitoring

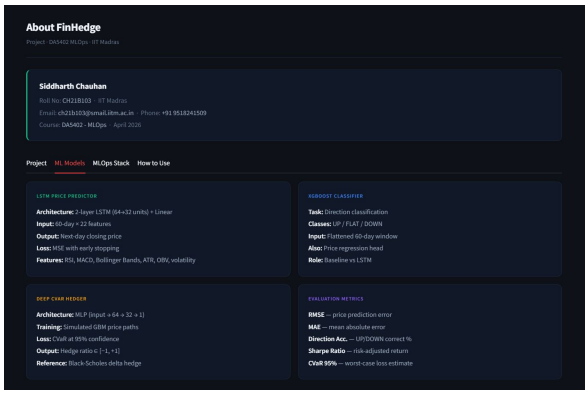


(a) Metrics tab - summary cards

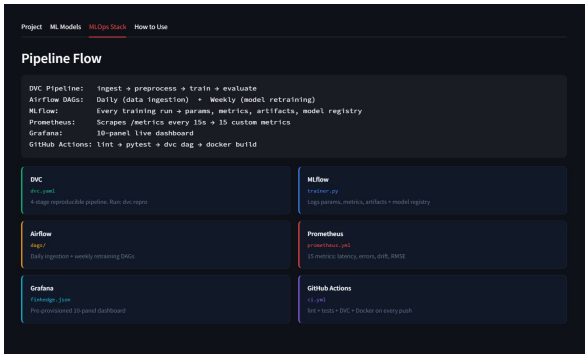


(b) Prediction rate timeseries

Figure 6: Page 5 - Monitoring (Metrics tab)

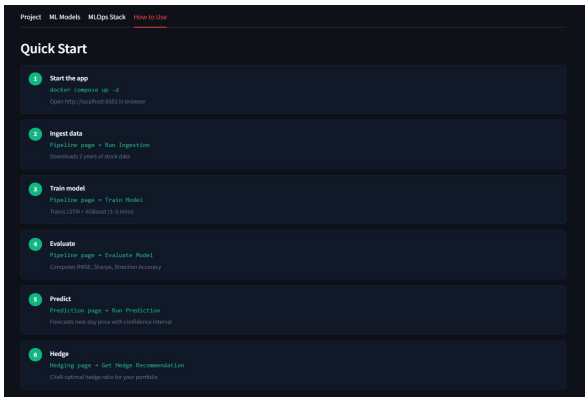


(a) Inference latency percentiles

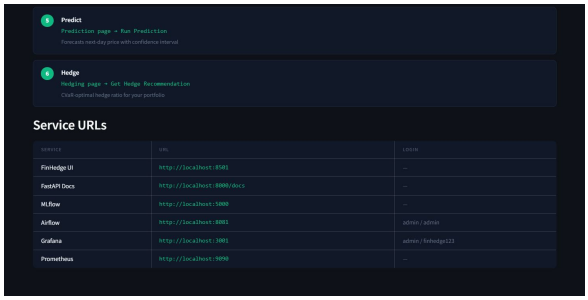


(b) Hedge action distribution

Figure 7: Page 5 - Monitoring (latency and hedge actions)



(a) Data drift detection tab



(b) Drift scores by feature

Figure 8: Page 5 - Monitoring (Drift Detection tab)

The Monitoring page has three tabs:

Metrics tab Shows live Prometheus-scraped metrics: total predictions, error rate, model RMSE, Sharpe ratio, prediction rate timeseries, latency percentiles (p50/p90/p99), and hedge action distribution.

Drift Detection tab Displays KL-divergence scores for 10 features vs. training baseline. Red bars indicate features above the drift threshold (0.05). Use the **Compute Drift** button to refresh scores via the backend endpoint.

Grafana Dashboard tab Provides a direct link to the Grafana dashboard with all 10 panels.

Enabling auto-refresh:

1. Use the sidebar **Auto-refresh** dropdown
2. Select 10s, 30s, or 60s interval
3. The page will auto-reload at the selected interval

Tip

To populate drift scores, navigate to `http://localhost:8000/docs`, find `POST /pipeline/drift/compute`, enter your ticker, and click **Execute**. The scores will appear on the Monitoring page immediately.

3 Running the ML Pipeline from CLI

You can also interact with the pipeline directly via the FastAPI REST API or using Python scripts.

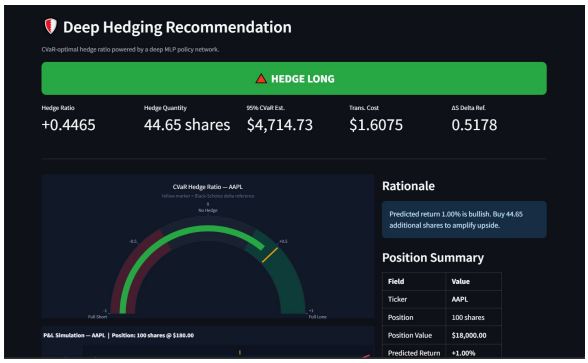
3.1 Using the REST API (curl)

Listing 1: Trigger full training via curl

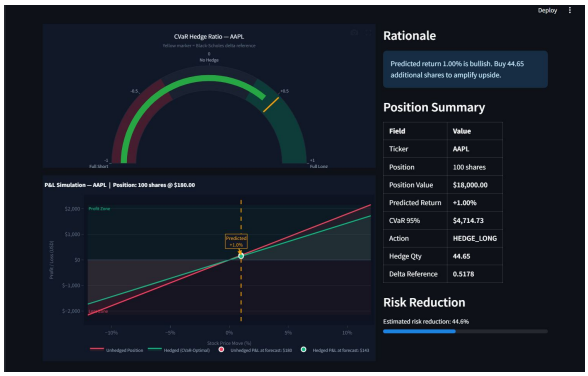
```
# 1. Ingest data for AAPL
curl -X POST "http://localhost:8000/pipeline/trigger" \
  -H "Content-Type: application/json" \
  -d '{"ticker": "AAPL", "stage": "ingest", "period": "2y"}'

# 2. Train LSTM model (synchronous - waits for completion)
curl -X POST "http://localhost:8000/pipeline/train" \
  -H "Content-Type: application/json" \
  -d '{
    "ticker": "AAPL",
    "model_type": "lstm",
    "epochs": 50,
    "batch_size": 32,
    "lr": 0.001,
    "period": "2y"
  }'

# 3. Get a prediction
curl -X POST "http://localhost:8000/predict/price" \
  -H "Content-Type: application/json" \
  -d '{"ticker": "AAPL", "model_type": "lstm"}'

# 4. Compute drift scores
curl -X POST "http://localhost:8000/pipeline/drift/compute?ticker
    =AAPL"
```

3.2 Using the FastAPI Swagger UI

1. Open http://localhost:8000/docs
2. Find the endpoint you want to call
3. Click **Try it out**
4. Fill in the request body and click **Execute**
5. The response appears below



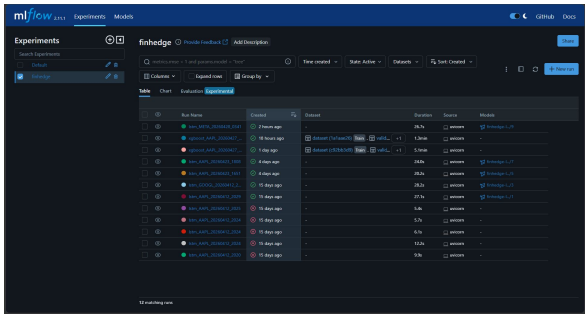
(a) Swagger UI overview



(b) Executing an endpoint

Figure 9: FastAPI Swagger UI at `/docs`

4 MLflow Experiment Tracking



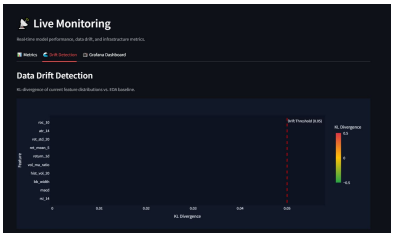
(a) MLflow experiment list

(b) Run details with metrics and artefacts

Figure 10: MLflow tracking server at `http://localhost:5000`

1. Open `http://localhost:5000`
2. Click on the **finhedge** experiment
3. Each training run appears as a row showing:
   - Duration, start time, run name
   - Metrics: RMSE, MAE, MAPE, R^2, Direction Accuracy, Sharpe Ratio
   - Parameters: ticker, model type, epochs, batch size, learning rate
   - Artefacts: saved model `.pt` file
4. Click any run to drill into details and compare runs
5. Use the **Compare** feature to plot metrics across multiple runs

(a) Run metrics view

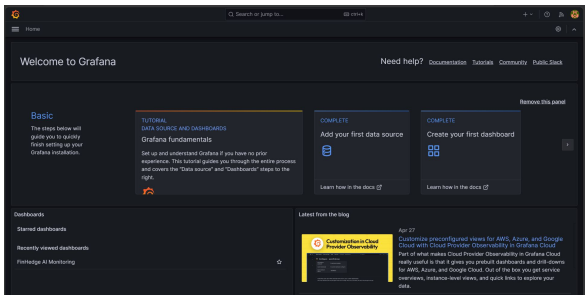


(b) Model artefacts

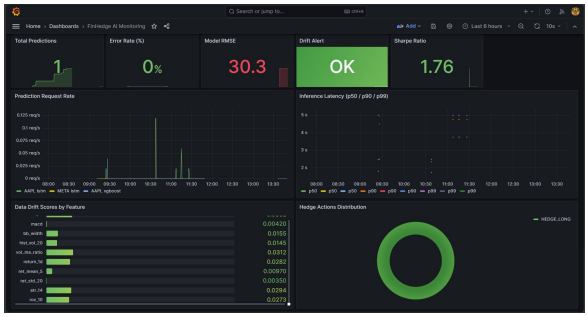
(c) Model registry

Figure 11: MLflow run details and model registry

5 Grafana Monitoring Dashboards



(a) Grafana - stat panels row



(b) Grafana - timeseries and bargauge

Figure 12: Grafana FinHedge AI Monitoring dashboard

1. Open `http://localhost:3000`
2. Login: **admin** / **finhedge123**
3. Click **Dashboards** → **FinHedge AI Monitoring**
4. The dashboard auto-refreshes every 10 seconds

Panel descriptions:

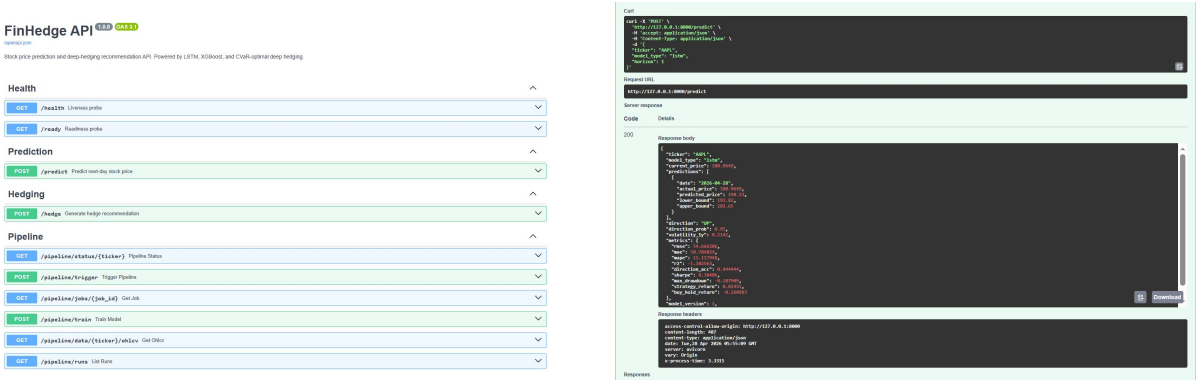
Table 2: Grafana dashboard panels

Panel	What it shows
Total Predictions	Cumulative prediction requests
Error Rate (%)	Prediction error rate (red above 5%)
Model RMSE	Current RMSE gauge (green <5, red >10)
Drift Alert	Binary OK / DRIFT indicator
Sharpe Ratio	Latest Sharpe (green >1, red <0.5)
Prediction Rate	Requests/second timeseries (last hour)
Inference Latency	p50, p90, p99 latency percentiles
Drift Scores	Horizontal bar gauge per feature
Hedge Distribution	Donut chart of LONG / SHORT / HOLD actions
Pipeline Status	Success/failure rate per stage

Note

Panels show "No data" until the corresponding pipeline stages have run at least once. Run the full training pipeline and make a few predictions first.

6 Apache Airflow - Scheduled Pipeline



(a) Airflow DAG list (b) DAG graph and task logs

Figure 13: Apache Airflow at `http://localhost:8080`

1. Open http://localhost:8080
2. Login: **admin** / **admin**
3. Find the **finhedge_pipeline** DAG in the list
4. Toggle the DAG **on** (click the switch on the left)
5. To trigger manually: click the **play** button → **Trigger DAG**
6. Click on a run to see the graph view and task status
7. Click any task → **Log** to see detailed logs

Tip

The DAG is scheduled to run daily at midnight UTC. You can change the schedule in `dags/finhedge_pipeline.py` (default: `schedule_interval="@daily"`).

7 Stopping and Restarting the Application

Listing 2: Container lifecycle commands

```
# Stop all containers (data preserved in volumes)
docker compose stop

# Stop and remove containers (data preserved in named volumes)
docker compose down

# Stop and remove everything including volumes (WARNING: deletes
    data)
docker compose down -v

# Restart a single service (e.g., after code change)
docker compose restart backend

# View logs for a service
docker compose logs -f backend

# Rebuild an image after code changes
docker compose build backend
docker compose up -d backend

# Check resource usage
docker stats
```

8 Troubleshooting

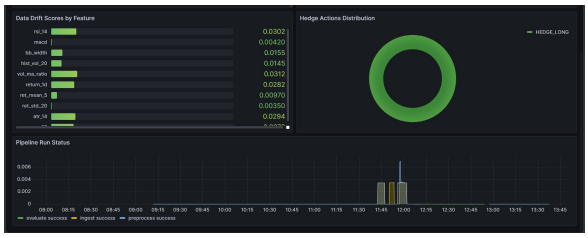
Table 3: Common issues and solutions

Symptom	Solution
`docker compose up` fails with port conflict	Another service uses the same port. Edit `docker-compose.yml` to change the host port (left side of `PORT:PORT`).
Streamlit shows "Connection refused"	Backend may still be starting. Wait 30s and refresh. Check: `docker compose logs backend`.
"Model not found" on Predictions page	Run the pipeline first. Go to Page 4 → Pipeline → click **Run Full Training Pipeline**.
Grafana panels show "No data"	(1) Run the pipeline and make predictions. (2) Check Prometheus is scraping: open `http://localhost:9090/targets`.
Drift scores all zero	Call the drift compute endpoint: `POST /pipeline/drift/compute?ticker=AAPL` via Swagger UI.
MLflow shows no runs	Training logs to the `finhedge` experiment. Check `docker compose logs mlflow`.
Airflow DAG not visible	Copy the DAG file: `docker cp dags/ finhedge_airflow-webserver_1:/opt/airflow/` then restart: `docker compose restart airflow-webserver`.
Training fails with CUDA error	The backend runs on CPU by default. This is expected - no GPU required.
`git pull` opens vim editor	Type `:wq` and press Enter to accept the merge commit message.

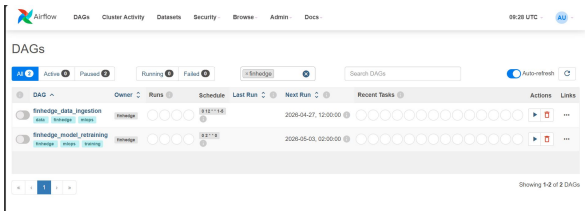
8.1 Checking Service Health

Listing 3: Health check commands

```
# Backend health endpoint
curl http://localhost:8000/health

# Prometheus targets (should show finhedge backend as UP)
open http://localhost:9090/targets

# All container statuses
docker compose ps

# Recent error logs
docker compose logs --tail=50 backend
docker compose logs --tail=50 mlflow
```



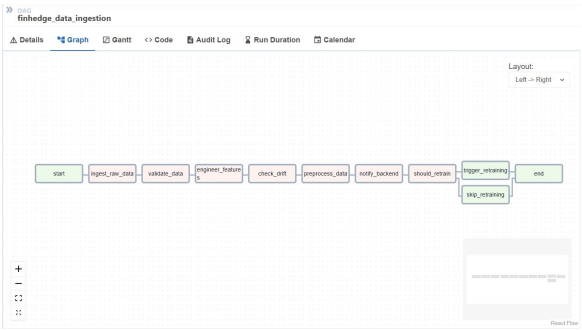
(a) Prometheus targets page



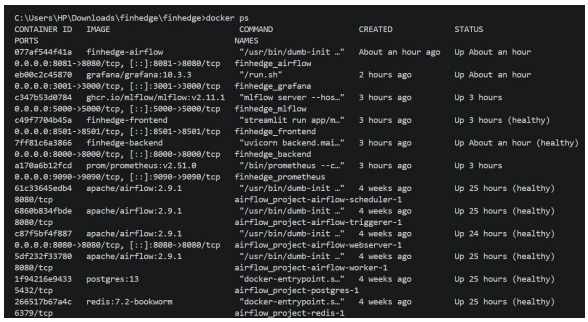
(b) Streamlit monitoring metrics

Figure 14: Health check views

9 CI/CD with GitHub Actions



(a) GitHub Actions workflow list



(b) CI job execution details

Figure 15: GitHub Actions CI/CD - runs on every push to `main`

Every `git push` to `main` or `develop` automatically triggers:

1. **Lint & Test** - flake8, black, isort, and pytest
2. **DVC Pipeline Check** - verifies DAG integrity
3. **Docker Build** - builds images and smoke-tests `/health`

To push code changes:

```
git add backend/ frontend/ dags/
git commit -m "feat: improve LSTM training convergence"
git push origin main
# CI runs automatically - check Actions tab on GitHub
```

10 Adding a New Ticker

To train a model for a new stock (e.g., `TSLA`):

1. Open the Streamlit app → **Pipeline** page
2. Change the ticker to `TSLA` in the sidebar

3. Click **Run Full Training Pipeline**
4. Wait for all four stages to complete (green indicators)
5. Switch to the **Predictions** page and enter `TSLA`
6. Click **Predict Next Close Price**

Tip

Any ticker supported by Yahoo Finance works (e.g., `GOOGL`, `AMZN`, `NIFTY50.NS`, `BTC-USD`). Use the correct Yahoo Finance symbol.

11 Environment Configuration

All configuration is in `docker-compose.yml` environment sections. Key settings you may want to change:

Listing 4: Common configuration overrides

```
# backend service
MLFLOW_TRACKING_URI: http://mlflow:5000
DEFAULT_SEQUENCE_LEN: 60        # LSTM lookback window (days)
DEFAULT_EPOCHS: 50              # training epochs
DEFAULT_HIDDEN_SIZE: 128        # LSTM hidden units

# frontend service
BACKEND_URL: http://backend:8000
PROMETHEUS_URL: http://prometheus:9090

# grafana service
GF_SECURITY_ADMIN_PASSWORD: finhedge123  # change in production!
GF_SERVER_ROOT_URL: http://localhost:3000

# airflow
AIRFLOW__CORE__EXECUTOR: SequentialExecutor
```

Warning

Never commit real credentials to version control. Use a `.env` file and add it to `.gitignore` for production deployments.

11 Appendix - Keyboard Shortcuts

Table 4: Useful keyboard shortcuts

Shortcut	Action
`Ctrl+C` (in terminal)	Stop `docker compose logs -f`
`Ctrl+R` (browser)	Hard-refresh Streamlit page
`S` (Grafana)	Save dashboard
`F` (Grafana)	Toggle fullscreen on a panel
`E` (Grafana)	Edit a panel
`Ctrl+Z` (Airflow)	Undo in DAG code editor